

H2O: Heavy Hitter Oracle for Efficient Generative Inference of Large Language Models

Abstract

In autoregressive generation, caching previous key and value states (KV-cache) prevents redundant computation. However, KV-cache size grows linearly with sequence length and batch size, becoming the main memory bottleneck during long-context LLM deployment. We present H2O (Heavy Hitter Oracle), a dynamic KV-cache eviction algorithm based on the observation that attention scores exhibit strong sparsity dominated by a small set of "heavy hitter" tokens.

1. Introduction

During generation, the memory footprint of the Key-Value (KV) cache frequently exceeds the parameter footprint of the model itself. For instance, serving a 30B model with context length 8k at batch size 32 requires more than 60GB of VRAM solely dedicated to KV caches.

2. Approach: Heavy Hitter Eviction Policy

H2O identifies that over 90% of attention volume across all layers concentrates on less than 15-20% of tokens (termed Heavy Hitters, H_{20}).

- Accumulative Attention Tracker: For each token in the context, H2O tracks the cumulative attention score assigned to it across previous decoding steps.
- Budget Allocation: Under a fixed KV cache budget B , H2O dynamically retains:
 1. The most recent local tokens (e.g., last 32 tokens) for local syntactic fluency.
 2. The top k heavy hitter tokens with the highest cumulative historical attention.
- Eviction: Low-scoring non-local tokens are permanently evicted from the GPU cache.

3. Empirical Findings

- Memory Reduction: Reduces KV cache memory footprint by up to 5x with negligible degradation in perplexity and downstream accuracy (tested on LLaMA-7B/13B/33B and OPT models).
- Throughput Improvement: Enables 3x to 4x higher batch sizes and 2.5x higher serving throughput on a single A100 GPU.
- Long Context Retention: Preserves document retrieval capability on LongBench and needle-in-a-haystack tasks up to 32k context lengths.

4. Limitations

- Irreversible Eviction: Once a token is dropped from the cache, it cannot be recovered if subsequent generation steps require it.
- Accumulated Score Overhead: Requires maintaining tracking buffers for cumulative attention weights per head.